

Git & Github & GitClassroom

Git

“Git是一种免费的，开源的分布式版本控制系统”

简而言之，是用来管理本地代码开发&多人项目合作的软件，可以很方便的进行版本控制。

那么版本控制是什么呢？游戏的Save&Load，顺便还会记录你每个存档之间的区别是什么（代码修改情况 == 任务完成情况）

本学期可能的使用场景

- 🙌🤔，我想给代码加个新功能，但是出bug了还把之前代码咋写的忘了
- 🙌🤔，我在好几个服务器上改了好多代码，要合并了对着几百行杂乱修改发愁
- 🙌🤔，我和其它同学在一块写代码，但是他发过来的文件我都没找到改了什么
- and so on.....

看吧，学校内尚且如此，之后只会越用越多，一定要好好学git

shell

Git是一种使用shell的程序，因此大家在linux终端，mac终端，windows终端以及powershell中均可以使用，但是由于windows终端的指令和剩下几种差距过大，故不做推荐。先前没有用过git的同学们尽量还是使用wsl（可以避免很多版本问题），如果用不了wsl/wsl安不了git的同学可以选择windows+git bash or powershell

```
$猜猜这个windows终端的cd命令咋用
C:\Users\username>cd d:\

C:\Users\username>cd /d:D
文件名、目录名或卷标语法不正确。

C:\Users\username>cd /d
文件名、目录名或卷标语法不正确。

C:\Users\username>cd D:
d:\

C:\Users\username>cd /d d:    ($正确答案)

d:\>
```

我的仓库在哪里——init

刚下下来的时候肯定是还没有仓库的，所以我们先尝试在本地进行一个git的初始化

```
$ mkdir git_test
$ cd git_test
$ git init
Initialized empty Git repository in D:/AI-learning/git_test/.git/
```

ok, 如果这里没有问题的话, 恭喜同学们进入了git世界的大门!

在正式体验git之前, 我们先讲一些概念性的东西, 这样大家会更方便理解后面命令操作之间的区别 (add是什么, 为什么还要commit?)

一个Git仓库可以分为四个部分

- 工作区: 我们在文件夹里的代码, 能看到的都算是工作区
- 暂存区: 暂时保存我们**提交的**修改, 是代码修改提交之前的暂存处, 一般在.git的index文件, 在commit之前不会写入版本库
- 本地版本库: .git文件夹, 保存完整的分支情况
- 远程版本库: github等等

一次**完整的修改提交流程**是: 工作区修改代码 ——> 使用add命令将修改加入暂存区 ——> 使用commit将暂存区的修改同步到版本库

所以说**直接commit是没有用的**, 同时你commit的也只有你自己**暂存的**修改

git的完整命令可以查阅[Git - Reference \(git-scm.com\)](https://git-scm.com)

一些简单操作——add&commit&status

首先, 我们先创建一个文件

```
$ touch a.txt
```

运行这个指令可以创建一个名为a的txt文件, 这时候我们运行

```
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    a.txt

nothing added to commit but untracked files present (use "git add" to track)
```

下面的显示告诉我们, 没有commits提交, 存在一个untracked file (未暂存的文件), 这时候我们使用git add进行提交

```
$ git add a.txt
```

当需要一次性添加很多文件到暂存区的时候, 可以使用一些关键字进行全部暂存

```
$ git add -A
```

这时候我们可以进入文件修改一些文字

```
$ echo "add someting interesting to this file" > a.txt
$ git status
On branch master
```

```
No commits yet
```

```
Changes to be committed:
```

```
(use "git rm --cached <file>..." to unstage)
```

```
new file:   a.txt
```

```
Changes not staged for commit:
```

```
(use "git add <file>..." to update what will be committed)
```

```
(use "git restore <file>..." to discard changes in working directory)
```

```
modified:   a.txt
```

可以看到这里 to be committed 中出现了我们刚刚添加的文件，同时下面也提示我们a.txt出现了修改，可能需要用add重新update一下。

这里我们add一下，之后直接commit

```
$ git commit -m 'Update a.txt'
[master (root-commit) 6aa9c58] Update a.txt
1 file changed, 1 insertion(+)
create mode 100644 a.txt
```

这里的-m关键字是必须的，后面是对于此次commit的描述

使用git log查看提交记录

```
commit 6aa9c58170be3890c1e1c4fcfac1e0e979f6c114 (HEAD -> master)
Author: username <useremail.com>
Date:   Thu Sep 12 14:00:26 2024 +0800

    Update a.txt
```

使用--oneline关键词可以压缩显示

我不想add这个——.gitignore

这个文件的作用是配置那些你不想管理的分支，比如说在写模型评测代码的时候肯定不想把100+mb大小的模型给add进去，这时候可以创建.gitignore文件来解决这个问题，比如下面这样的文件

```
# ignore experiments
experiments/
# ignore python cache
__pycache__/
```

作用就像注释里写的一样，会在commit的时候忽略这两个文件夹，是十分有用的文件，同时也支持表达式匹配的方式

更多详情请查看[Ignoring files - GitHub Docs](#)

版本回退——reset

这个大多数情况下用的不多，但是作为“Load”还是要讲一下。使用reset指令，我们可以回退到我们其中一个commit记录处。使用git reset --help可以查看更多的使用方法，这里只讲一下怎么进行版本回退

首先，使用git log查看我们的提交记录

```
$ git log
commit a12c545115e140f2698349a4598fdb4c45c3c943 (HEAD -> master)
Author: username <useremail@gmail.com>
Date: Thu Sep 12 14:19:55 2024 +0800

    wrong commit

commit 6aa9c58170be3890c1e1c4fcfac1e0e979f6c114
Author: username <useremail@gmail.com>
Date: Thu Sep 12 14:00:26 2024 +0800

    Update a.txt
```

接下来要想办法跳回最初的commit，观察到这串诡异数字，这个是版本号。通过版本号跳转

```
$ git reset --hard 6aa9c58170be3890c1e1c4fcfac1e0e979f6c114
HEAD is now at 6aa9c58 Update a.txt
```

回退完成

多线程的并行——branch&checkout

在很多时候，我们想要尝试性的给我们的代码增加一些新的功能，但是如果一直在原代码上写，写错了再回退又显得很麻烦。这时我们不妨另开一个同样的复制在上面随意的捣鼓代码，做好了再同步回去不就可以了，这也就是分支branch的思想，使用branch指令创建全新分支并切换。

```
git branch main
git checkout main
```

接下来就可以放飞自我的对代码进行修改了

收菜——merge

既然我们在分支上已经完成了功能的实现，是时候把它同步到主干分支main上了，但是应该怎么同步？CV大法过于麻烦，而且对于两个都做了修改的分支无能为力，这时候就应该使用merge指令对分支进行合并了。

merge指令常用于合并两个分别具有修改的分支为一个分支，具体操作如下

```
git merge <branch>
```

这个指令会将branch合并到当前分支

```
$ git merge master
Updating 6aa9c58..77653b9
Fast-forward
 a,txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 a,txt
```

Github

接下来通过一个实际的github案例向大家展示一下github的一些工作流程以及git的剩余几个操作

Start a new repository for xushaoxuan123

A repository contains all of your project's files, revision history, and collaborator discussion.

Repository name *

☐ Public

Anyone on the internet can see this repository

☒ Private

You choose who can see and commit to this repository

Create a new repository

像这样先创建一个项目

怎么把文件推到github——remote&push&pull

接下来通过将本地的仓库连接到github的repo，以进行后续的所有操作

```
git remote add origin https://github.com/Yourname/git_test.git
```

接下来，我们将现有的仓库上传到github这个托管平台上

```
$ git push origin master
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:      https://github.com/Yourname/git_test/pull/new/master
remote:
To https://github.com/Yourname/git_test.git
 * [new branch]      master -> master
```

指令中origin是远程仓库的默认名称，master是你想推送分支的名称

如果远方仓库的内容有所更新，可以使用pull命令拉取仓库内容

```
git pull origin main
```

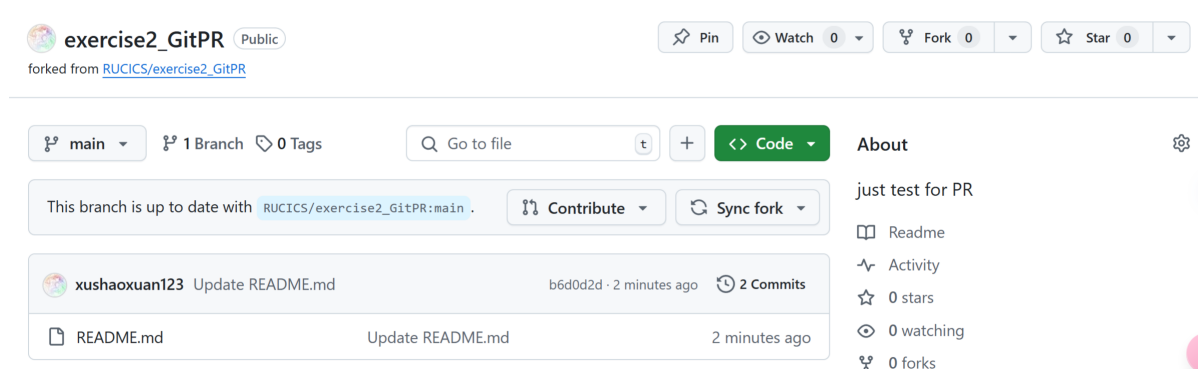
注意：push操作在多人协作中是非常危险的操作，尤其是直接推送到主分支的操作，稍有不慎就会导致main branch报废，同时也要谨慎使用merge操作，切勿强行merge

PR是啥东西（1）——fork&clone

fork 是指在某个现有的代码仓库的基础上创建一个属于你自己的副本。这个副本是原仓库的一个完整拷贝，包括所有的代码、分支和提交历史。

一般用来在别人的代码基础上修修改改，也是提出PR（Pull Request）的前提。

首先fork一个仓库



然后我们把它clone下来，注意如果ssh没有配置好的话用ssh的链接可能会出问题

```
git clone https://github.com/xushaoxuan123/exercise2_GitPR.git
```

注意这里后面的链接别复制过去，不然之后是提交不了PR的

PR是啥东西（2）——PR&review

然后我们新建分支并对文件进行一下修改，然后提交到我们的repo里，可以看到repo里面出现了PR按钮

test_1 2 Branches 0 Tags Go to file + <> Code

This branch is 1 commit ahead of RUCICS/exercise2_GitPR:main .

Contribute Sync fork

This branch is 1 commit ahead of RUCICS/exercise2_GitPR:main .
Open a pull request to contribute your changes upstream.

Open pull request

e3a79e6 · 5 minutes ago 3 Commits

add a line to file 5 minutes ago

用来进行测试的Readme，同学们可以对其进行修改后发起PR请求
比如说我在这里加入一行新的并提交

进行提交之后，会出现这样的界面

Open Test it before exercise2 #1
xushaoxuan123 wants to merge 1 commit into RUCICS:main from xushaoxuan123:test_1

xushaoxuan123 commented 4 minutes ago Member

这里一般是写对PR的描述

add a line to file e3a79e6

Require approval from specific reviewers before merging
Rulesets ensure specific people approve pull requests before they're merged. Add rule

Continuous integration has not been set up
GitHub Actions and several other apps can be used to automatically catch bugs and enforce style.

✓ This branch has no conflicts with the base branch
Merging can be performed automatically.

Merge pull request You can also open this in GitHub Desktop or view command line instructions.

如果没有问题，当authors进行code review之后就可以将PR合并到主分支了。打开仓库的Pull Request即可进行review

Finish your review

Write

Preview

H

B

I

≡

<>

🔗

≡

≡

≡

📎

@

🗨️

↩️

Leave a comment

📄 Markdown is supported

📎 Paste, drop, or click to add files

☐ Comment

Submit general feedback without explicit approval.

☒ Approve

Submit feedback and approve merging these changes.

☐ Request changes

Submit feedback that must be addressed before merging.

Submit review

我要提bug——issues

issues有相当多的功能，可以提问题，做规划，设目标等等，不过最主要的功能还是提出、讨论问题。对于开源项目而言，官方文档里都会有写具体的issues格式，如果你不按照对应格式提交很有可能直接被close掉，比如metagpt的提交bug的issues格式如下

```
**Bug description**
<!-- Clearly and directly describe the current bug -->

**Bug solved method**
<!-- If you solved the bug, describe the idea or process to solve the current bug. Of course, you can also paste the URL address of your Pull Request. -->
<!-- If not, provide more auxiliary information to facilitate our further positioning and investigation -->

**Environment information**
<!-- Environment: System version (like ubuntu 22.04), Python version (conda python 3.7), LLM type and model (OpenAI gpt-4-1106-preview) -->

- LLM type and model name:
- System version:
- Python version:
- MetaGPT version or branch:

<!-- Dependent packages: the packages version cause the bug(like `pydantic 1.10.8`), installation method (like `pip install metagpt` or `pip install from source` or `run in docker`) -->

- packages version:
- installation method:
```

```
**Screenshots or logs**
```

```
<!-- Screenshots or logs of the bug can help us understand the problem more quickly -->
```

GitHubClassroom

GitClassroom是本学期提交代码作业的平台，同时配备有自动评分系统。助教们会在每次作业发布之后将链接发送到群里，同学们接受邀请后即可进行作业的写作。

大家可以查看文档进行详细了解[查看自动评分结果 - GitHub 文档](#)

Vscode中的拓展

Vscode中有着众多的git相关拓展，可视化提交的gitgraph，显示代码提交信息的git lens等等，这里主要推荐的是gitgraph和git lens。

更多的可以查看[增强 Git 的最佳 VS Code 扩展 \(zhihu.com\)](#)等相关博客